Computer Architecture and System Software

Lecture 05: Assembly Language cont.

Instructor:
David Bosc
Applied Computer Science
University of Winnipeg

Announcements

- Assignment 2 is out this week
  - Due Friday, Feb 20th (during reading week)
  - After today's lecture, should be able to complete the assignment
    - Q1 & Q2: Frac
      - Be sure to show your work in binary where applicable for full marks
    - Q3: Assembly Programming
- No lecture next week, as it is the midterm exam
  - Details next slide section
  - Open lab for As2 help if you want to get it done before the break

Midterm Info

Midterm Info

- February 9th/10th (depending on your section) @6pm-8pm in your lecture room
  - 2 hours to complete
- In person, written by hand **without** outside notes, calculators, handheld devices, etc.
- Worth 24% of final grade
- Covers lectures 1-3 (ie. no Assembly)
- Will have to show all your work, so bring pen, pencil, eraser.
  - Scrap paper will be provided

Ethical use of LLMs to study is recommended!

Studying - Tips

- Read the textbook
- Form a study group
- Email me questions if you're unsure how to answer
  - LLMs can be a great resource, but they won't be marking your tests
- Create personal reviews per lecture to have all content in once place, easy access
  - Read your reviews while you eat, before bed, break at work, etc.

I/O in Assembly

Interrupt Instructions

- INT operand
  - Operand could be an immediate value, which is an interrupt numbered by immediate byte (0 .. 255)
  - Push to stack: flags, registers, CS, IP, IF = 0 and transfers control to interrupt procedure
- IRET – return from an interrupt service routine

- Although there are 255 possible operands, the one we will use **21h** exclusively in this course. This transfers control to the operating system to allow the OS to run a subroutine.

Interrupt Instructions

- Summary:
  - Sometimes we need the operating system to call a subroutine (ie. read user input, print out results)
  - The OS is the interface between our program and the hardware
  - We use the **int 21h** instruction to call the operating system which executes a subroutine based on the code stored in the **ah** register
- For our example, if we want to read a character we need to run subroutine #1, which has the following requirements

Subroutine 1

- **Subroutine 1** reads one char of input from the user

- Waits for user input at command line
- When user presses a key, that character is stored in register **al** (using ASCII)
- Simplest form of user input. No user prompting

```
mov ah, 1        ;Subroutine code
int 21h          ;Call OS
```

Subroutine 2

- **Subroutine 2** displays the character in register **dl**
- Subroutine decodes number using ASCII
- Prints to command line

```
mov dl, 34h        ;We want to display a char from
                       register dl
mov ah, 2h         ;Give the subroutine code
int 21h            ;Call the OS
```

Subroutine 9

- **Subroutine 9** displays multiple chars (strings)
- Subroutine 9 looks at the **memory address** specified by the contents of register **dx**
- The subroutine prints the contents of each subsequent memory address until it hits the terminating '**$**' character.

Strings in Assembly

- Recall that strings can be defined in the following way

  mystring db 'Hello World.'

- For the following example, let's modify this line slightly:

  mystring db 'Hello World.', 10, '$'

- **Line-feed** (ASCII 10) is the control character that brings the cursor down to the next line in the terminal (\n)
- Other machines require a **Carriage-Return** (ASCII 13D) before the line-feed
- '$' is a terminating character

Recall: Data transfer instructions

lea register, memory_address

- Examples

lea dx, mystring

- Result: dx contains the starting memory address of string 'mystring'
- mystring is ready to print (if it is terminated)

HelloWorld.asm

- Let's take a look at HelloWorld.asm to see the subroutine in action
- Let's also try modifying to display a second string on a new line.
- Try editing specific characters within the string using assembly instructions.

AdditionExample.asm

- Let's use our knowledge of subroutines and display the results of addition

- Initialize two variables var1, var2 as 2 and 4.
- Add them together.
- Print the results.

How to interpret output

- Displaying the number 6 returns ♠
  - Why?

How to interpret output

- Displaying the number 6 returns ♠
  - Why?
  - ♠ has the ASCII encoding of 6
  - If you are on Windows you can type characters from the ASCII table by typing Alt+num at command line
- So how do we display the character '6'?
  - The characters '1','2',…'9' all appear in the ASCII table with offset of 48 (or 30h).
  - Therefore adding **30h** to our register before displaying should fix our output

How to interpret output

- Let's try adding 5+5. What happens?

- Let's subtract 2-4. What happens?

How to interpret output

- Let's try adding 5+5. What happens?
  - There is no ASCII entry for '10'. Each digit has to be decoded separately. In general, converting multi-digit base 10 numbers to string format is a complicated process that we'll look into next class.

- Let's subtract 2-4. What happens?
  - 0x02-0x04 = 0xFE. Adding the 30h offset results in overflow. 0xFE+0x30 = 0x2E or $(46)_{10}$.
  - This is the ASCII code for '.'

Warning(!!)

- Sometimes **int 21h** may overwrite the **al** register
- Preserve the al/ax register by
  - Copying to another register
  - Copying to memory (you can reserve unallocated space in the data segment)
  - Pushing to the stack (more on this later)
- If unsure, use the debugger to trace through

Assembly Template

Preliminaries

- Assembly instructions are not case sensitive
- Comments are preceded by ‘;’
  - ;This is a comment
- No terminating character – each instruction is on its own line. Operands of instructions separated by comma
  - mov ah, 9 ;first instruction
    add ah, 9 ;second instruction
    …
- Code should be heavily commented, since ASM is not easily readable

Template

- I have already provided a template for you online (template.asm). Let's step through it slowly to see what is going on.

Data Segment

- The data segment will hold all our variable declarations
  - Begins with **data segment**
  - Ends with **data ends**
  - Any variable declarations/memory allocations are made in between these two statements
  - We will not worry about dynamic memory allocation in this course

Stack Segment

- Before any instructions we need to define the stack segment
  - stack1 segment stack starts the stack segment
    - stack is a reserved name so we call the stack 'stack1'
  - stack1 ends ends the stack segment
  - We will allocate 100 uninitialized bytes – for this course we will never need more than this
    - db 100 dup(?)

Code Segment

- Start and end of the code segment:

```
code  segment
code  ends
```

- Our instructions will be written between these statements

- Our first instructions need to name the various segment registers and assign their locations in mem

```
assume cs:code, ds:data, ss:stack1
```

Template

- Now point the data and stack registers to their proper segments

```
mov ax, data
mov ds, ax

mov ax, stack1
mov ss, ax
```

Template

- Our next instruction: Start:
- The colon after 'Start:' defines it as a **label**
- In the future, we'll use labels to jump ahead or backwards in our code
- The first label is required to tell the assembler your code has begun. The label can be called anything (start, begin, etc)

Template

- We'll end the label immediately after the code segment (normally we will not end labels)

```
code ends
end start
```

Template

- Now we want to return control back to MS-DOS to end the program with the subroutine code 4Ch, which we'll store in the ah register

```
mov ah, 4ch
int 21h
```

Addressing Modes

x86 Addressing Modes

- Specify the method for finding the memory address of an operand
  - e.*g*. using information held in registers and/or constants contained within a machine instruction
- Specifies operands in an assembly language program and is completely architecture dependent

Recall MOV

MOV AH, 08h

- h stands for hexadecimal
- MOV copies operand 2 into operand 1
  - Source (data value 08h) is copied into destination (register AH)
- Data 08h is provided in the instruction
  - Called Immediate addressing mode

x86 Addressing Modes

- There are three basic types of operands:
  - Immediate
    - Constant integer (8, 16, or 32 bits)
    - Constant value is stored within the instruction
  - Register
    - Name of a register is specified
    - Register number is encoded within the instruction
  - Memory
    - Reference to a location in memory
    - Memory address is encoded within the instruction, or
    - Register holds the address of a memory location

x86 Addressing Modes

- The Intel 8086 has about 9 data addressing modes:
  - Immediate, register, direct, register indirect, based, indexed, based indexed, string, and port addressing
    - Definition names vary across books
  - Register and immediate modes do not need to access the external bus (faster)
  - The syntax of the operand field determines the addressing mode

Immediate Addressing Mode

MOV CX, 10

- Immediate data is coded directly in the instruction's machine code
  - Note: CX contains 0xA because there is no h after 10
- The data is put in the operand field
- The constant in the operand field may be of byte or word length
- Some assemblers need a '#' before the constant

Immediate Addressing Mode Cont.

- The constant may be in hex., dec., binary, or text
  - Default format is decimal
- Examples
  - MOV CX, 65
  - MOV CX, 41h
  - MOV CX, 01000001b
  - MOV CX, 'A'
- Advantage: No memory reference
- Disadvantage: Size of number is restricted to size of address field

Direct Addressing Mode

MOV DS:[1234h], AL

- Copies AL using offset 1234h
- The operand is stored in a MEMORY location
  - DS specifies data segment register
- An offset address is coded directly in the instruction
- The offset combined with DS forms the address where the operand is located

Direct Addressing Mode Cont.

- Ordinarily, DS: is assumed unless explicitly overridden using a colon (e.g. ES:[001h])
- Advantage: Single memory reference to access data
- Disadvantage: Limited address space

Direct Addressing Mode Cont.

- Examples:
  - Assume DS=10000h, ES=20000h, FRED=4567h
  - MOV AX, [20h]; load the contents at address 10020h and 10021h into AL and AH respectively
  - MOV DS:[1234h], AL; copy AL to address 11234h
  - MOV ES:[1234h], AL; copy AL to address 21234h
  - MOV FRED, AL; copy AL to address 14567h

Register Addressing Mode

- Instruction gets its source data from a register
- Data can be either 8/16/32 bits in length
- Data resulting from the operation is stored in another register
- Advantages:
  - Only a small address field is needed in instruction
  - No memory references are required
- Disadvantage: address space is very limited

Register Addressing Mode Cont.

- Examples:
  - MOV AX, BX; copy the 16 bit content of BX to AX
  - MOV AL, BL; copy the 8-bit content of BL to AL
  - MOV SI, DI; copy DI into SI
  - MOV DS, AX; copy AX into DS
- The following register to register transfers are not permitted.
  - MOV BL, BX; mixed sizes
  - MOV CS, AX; CS cannot be the destination
  - MOV ES, DS; Segment register to segment register forbidden

Register Indirect Addressing Mode

MOV CX, [BX]

- Copies a word from memory with location specified by BX
- Uses a register instead of a constant to specify the 16-bit offset address of the operand
- **Offset address can only be in the following registers: BP, BX, DI, SI.**
- The [ ] is needed to denote register indirect addressing mode
- The DS register is the default segment address register (except BP, which uses SS as the default register)

Register Indirect Addressing Mode

- In cases of ambiguity: assemblers need the presence of BYTE PTR or WORD PTR directives to indicate the size of the data addressed by the memory pointer
- e.g. MOV [DI], 10h is ambiguous since assembler does not know whether to save 10h to memory as a byte or a word
- Instead, use the following:
  - MOV BYTE PTR [DI], 10h; save 10h to memory
  - MOV WORD PTR [DI], 10h; save 0010h to memory

Register Indirect Addressing Mode

- Register indirect addressing is commonly used to access a table of data in memory
- Examples:
  - Assume BX=0222h, DS=10000h, SS=20000h, BP=0111h
  - MOV CX, [BX]; Copy a byte from address 10222h and 10223h to CX
  - MOV [BP], DL; Copy a byte from register DL to address 20111h
- Remember: Indirect addressing using BP defaults to SS

Based Addressing Mode

- Operand is located at the address given by adding 8 or 16 bit displacement to either BX or BP and combing the result with a segment register
- Displacement must be specified in the operand field
- Interpreted as signed 2's complement value
- Examples
  - Assume DS=10000h, SS=20000h, BP=0222h, BX=0111h
  - MOV AX, [BP-2]; Copy the content of 20220h and 20221h to AX
  - MOV [BX+777h], AX; Copy AL to 10888h and AH to 10889h

Indexed Addressing Mode

- Similar to based addressing except the index registers (SI or DI) must be used instead
- Operand is located at the address given by adding 8 or 16 bit displacement to either SI or DI and combing the result with a segment register
- Examples
  - Assume DS=10000h, SI=222h, DI=111h
  - MOV [DI-1], BL; store the content of BL to 10110h
  - MOV BX, [SI+1000h]; Load BL with the contents of 11222h and BH with the contents of 11223h

Indexed Addressing Mode Cont.

- Based and indexed addressing are aka REGISTER RELATIVE addressing
- Other syntax may be permitted to indicate the displacement
- e.g. the following examples are all equivalent
  - And they all result in the same assembled binary code
- Examples
  - Assume FRED is a constant defined in the assembly code
  - MOV [DI+FRED], BL
  - MOV [DI]+FRED, BL
  - MOV FRED[DI], BL

Based-Index-Relative Addressing

- The base and index registers are added to give the segment offset of where the operand is located
- The base register (either BX or BP) is added to an index register(DI or SI) as positive integers only
  - Each register lies in the range 0 to 65535
- By default, the segment address is derived from DS except the BP register, which is derived from SS
- A signed displacement may also be included to calculate the offset

Based-Index-Relative Addressing

- Examples
  - Assume SS=10000h, SI=3333h, BP=2222h
  - MOV AX, [SI+BP]; Load the content of 15555h and 15556h to AL and AH respectively
  - MOV AX, [SI+BP+1111h]; Load the contents of 16666h and 16667h to AL and AH respectively

String Addressing Mode

- A string is a series of bytes or words in sequential memory locations
- String instructions do not use any of the previous address modes
- Strings may be up to 64KB in length
- String addressing modes uses SI, DI, DS, and ES registers
- String instructions assume SI points to first byte of string to be processed, and DI points to first byte of destination string
- The use of a register my be implicit in the instruction

String Addressing Mode

- Example
  - Assume DS=10000h, ES=20000h, SI=10h, DI=20h
  - MOVSB; Move string byte from 10010h to 20020h

Port Addressing Mode

- 8086 has separate input/output space
- Up to 65536 I/O ports are available
- The I/O ports may be addressed by a byte sized constant
  - Limited to I/O ports in the range 0 to 255
  - IN AL, 40h; Put the content of I/O port 40h into AL
  - OUT 80h, AL; Send the contents of AL to I/O port 80h
- The I/O ports may be addressed using a register
  - Full range of 65536 ports are accessible
  - IN AL, DX; Load AL with the byte from port address given by DX
  - OUT DX, AX; Send the word in AX to port address given by DX

Summary of Addressing Modes

- Operand needed for an instruction may be located:
  - Immediately in the operand field, e.g. MOV AX, 1234h
  - In a register (register addressing), e.g. MOV DS, AX
  - In memory at a an offset specified by one of the following (disp is a constant):
    - [disp]
    - [BP]
    - [BX]
    - [SI]
    - [DI]
    - [BX+SI]
    - [BX+DI]
    - [BP+SI]
    - [BP+DI]
    - [BP+disp]
    - [BX+disp]
    - [SI+disp]
    - [DI+disp]
    - [BX+SI+disp]
    - [BX+DI+disp]
    - [BP+SI+disp]
    - [BP+DI+disp]
  - The segment address is in DS (by default, except when BP is used)
  - In memory locations given implicitly by string instructions
  - At input/output ports specified by a register or a constant

Buffered Strings

Buffered Keyboard Input

- Called with subroutine code 0Ah
- Reads an input string of characters that ends when the user presses enter
- Effective address of string should be stored in dx
- Before we can call the subroutine, we'll need space to store the input string

Strings for buffered keyboard input

- Normal string definition

```
myStr db 'Hello World!'
```

  - This reserves a byte for each character: 'H', 'e', 'l'…etc.
- Buffered string definition
  - `myBStr db 5, 6 dup(?)`
    - We specify the max number of characters we want to read (5 in this example)
    - We reserve 6 bytes for the 5 characters + enter key (also called carriage return)

Buffered Keyboard Input

- Memory structure of buffered keyboard input:

  max | count | Buffer

  1 byte |1 byte| N bytes

  where max = max number to characters read

  (including enter)

  count = number of characters returned

  (not including enter)

  (value generated by subroutine)

  Buffer = contents of string

  (provided by user)

Buffered String Overview

- Define buffered string in data segment
- Move code 10 (**0Ah**) to register **ah**
- **Load effective address** of buffered string in register **dx**
  (ie. dx contains address of 'max' byte)
- Call OS with **int 21h**

Don't forget the line feed!

- Once the user has entered a string, we need to make room on the command line for any other input/output for later.
- Use subroutine 2 to display the line feed character

```
mov ah, 2      Display subroutine
mov dl, 10     Line feed character
int 21h
```

Accessing memory

- Recall that we are only allowed to access memory using index or base registers
  - **Si, di, bx** index into **data segment**
  - **Bp** indexes into **stack segment**